Exercícios — Funções e Cifra de César em Python

Objetivo

Neste conjunto de exercícios, você irá construir uma **Cifra de César** utilizando funções definidas pelo usuário.

Você irá praticar:

- Criação de funções com `def`;
- Parâmetros e argumentos;
- Retorno de valores com `return`;
- Entrada de dados com `input()`;
- Laços `for`;
- Manipulação de strings;
- Conversão de tipos;
- Reutilização de funções.

Exercício 1 — Criar uma função

1. Crie a função

No seu arquivo Python, crie uma função chamada:

```
getDoubleAlphabet
```

Ela deverá receber um argumento chamado `alphabet`.

A função deve:

1. Receber o alfabeto;
2. Concatenar o alfabeto com ele mesmo;
3. Retornar o resultado.

> **Dica:** utilize `def` para criar a função e `return` para devolver o resultado.

Por exemplo, se o argumento recebido for:

```
ABC
```

o resultado deverá ser:

```
ABCABC
```

> **Importante:** o operador `+` também pode ser utilizado para concatenar strings.

Exercício 2 — Obter uma mensagem

Agora crie uma função chamada:

```
getMessage
```

Essa função deverá solicitar ao usuário uma mensagem que será criptografada.

Sua função deve:

1. Utilizar `input()` para solicitar a mensagem;

2. Armazenar a resposta em uma variável;

3. Retornar essa variável.

> **Dica:** diferentemente da função anterior, essa função não precisa receber nenhum argumento.

Exercício 3 — Obter a chave de cifra

Crie uma função chamada:

```
getCipherKey
```

Ela deverá solicitar ao usuário a **chave da cifra**.

A chave representa quantas posições cada letra deverá ser deslocada.

Utilize valores inteiros entre:

```
1 e 25
```

Sua função deve:

1. Solicitar a chave utilizando `input()`;

2. Armazenar o valor recebido;

3. Retornar a chave.

> **Atenção:** o `input()` retorna uma string. O tratamento necessário para utilizar a chave como número será importante nas próximas etapas.

Exercício 4 — Criptografar uma mensagem

Agora você irá criar a função responsável pela parte principal da cifra.

Crie uma função chamada:

```
encryptMessage
```

Ela deverá receber três argumentos:

```
message
cipherKey
alphabet
```

Antes de escrever o código

Pense no algoritmo seguindo esta sequência:

1. Criar uma variável para armazenar a mensagem criptografada;
2. Transformar a mensagem para letras maiúsculas;
3. Percorrer cada caractere da mensagem utilizando `for` ;
4. Descobrir a posição do caractere no alfabeto;
5. Calcular a nova posição somando a chave de cifra;
6. Utilizar a nova posição para descobrir a nova letra;
7. Adicionar a nova letra à mensagem criptografada;
8. Caso o caractere não esteja no alfabeto, mantê-lo sem alteração;
9. Ao terminar o `for` , retornar a mensagem criptografada.

Dicas importantes

Para encontrar a posição de um caractere dentro de uma string, você pode utilizar:

```
find()
```

Para acessar um caractere de uma determinada posição, utilize a indexação:

```
string[posição]
```

A chave recebida pelo `input()` também precisará ser tratada como um número durante o cálculo.

> **Atenção:** espaços e outros caracteres que não fazem parte do alfabeto não devem ser criptografados.

Exercício 5 — Descriptografar uma mensagem

Agora crie uma função chamada:

```
decryptMessage
```

Em vez de criar novamente toda a lógica da cifra, você irá **reutilizar a função `encryptMessage`**.

Para desfazer o deslocamento realizado durante a criptografia, a chave deverá ser utilizada de forma negativa.

Sua função deverá:

1. Receber a mensagem, a chave e o alfabeto;
2. Transformar a chave em um valor negativo;

3. Chamar `encryptMessage()` utilizando essa nova chave;

4. Retornar o resultado.

> **Objetivo:** evitar duplicação de código. A mesma lógica utilizada para criptografar será reutilizada para descriptografar.

Exercício 6 — Criar a função principal

Agora você irá criar a função que organiza todo o funcionamento do programa.

Crie uma função chamada:

```
runCaesarCipherProgram
```

Fluxo da função

Dentro dela, siga esta ordem:

1. Criar uma variável contendo o alfabeto inglês;

2. Exibir o alfabeto;

3. Criar o alfabeto duplicado utilizando `getDoubleAlphabet()`;

4. Exibir o resultado;

5. Obter uma mensagem utilizando `getMessage()`;

6. Exibir a mensagem;

7. Obter a chave utilizando `getCipherKey()`;

8. Exibir a chave;

9. Criptografar a mensagem utilizando `encryptMessage()`;

10. Exibir a mensagem criptografada;
11. Descriptografar a mensagem utilizando `decryptMessage()`;
12. Exibir a mensagem descriptografada.

> **Dica:** utilize as funções que você criou nos exercícios anteriores em vez de repetir suas lógicas dentro da função principal.

---

7. Execute o programa

Salve o arquivo e execute-o.

Inicialmente, pode parecer que **nada aconteceu**.

Isso acontece porque criar uma função não significa executá-la.

Você precisa **chamar a função principal** depois de sua definição.

Adicione uma chamada para:

```
runCaesarCipherProgram()
```

Depois execute o programa novamente.

---

8. Teste diferentes entradas

Experimente executar o programa utilizando:

- Mensagens com uma única palavra;
- Mensagens com várias palavras;
- Mensagens contendo espaços;
- Diferentes valores de chave;

- Letras maiúsculas e minúsculas.

Observe principalmente se:

- A mensagem é convertida para maiúsculas;
- Os espaços permanecem na mensagem;
- A mensagem criptografada muda de acordo com a chave;
- A mensagem descriptografada volta ao conteúdo original.